

CostVision and CAPEE

Two ways we could use the tool, what each one needs from us, and which one I would do first

1

Feed CAPEE automatically

CAPEE carries on doing the costing. CostVision reads the numbers off the 3D model and the drawing and puts them in.

2

Run CostVision on JLR rates

Install CostVision as it is. Take out the rates it comes with and put our own material, machine, labour and energy costs in.

SUMMARY

The two options

OPTION 1 · Feed CAPEE automatically

CAPEE stays as it is and still does the costing.
Instead of typing thirty boxes, the engineer hands over the 3D model and the drawing and the boxes fill in.

Moves about 5,700 lines of measuring and reading code, plus the safety checks that go with it.

OPTION 2 · Run CostVision on JLR rates

CostVision goes in as it is. Nothing gets rebuilt.
We load our own material prices, machine rates, labour and energy costs over the ones it ships with.

The way it calculates does not change. The work is gathering our rates and putting it on a JLR server.

WHAT I WOULD DO · Option 2 first, then Option 1

Option 2 is the smaller job and we can stop it whenever we want. It also tells us something Option 1 never will: whether the tool gets the right answer on JLR parts. So the rest of this deck goes through Option 2 first, then Option 1.

BEFORE WE DO EITHER

CostVision has never been checked against a price JLR has actually paid. Not one part. Until we load 30 to 50 real purchases and compare, nobody can say how accurate it is, and I would not sign either option off as ready for a supplier meeting. That job is ours, not the software.

CURRENT STATUS

Where CostVision is today

Taken from the software in August 2026. This is what the tool does today.

WORKING NOW

- 19 part types can be costed
- The costing gives the same answer every run
- Measures 3D models with no internet connection
- Reads drawings for tolerances, finish, heat treatment
- Safety checks on everything read off a drawing
- PDF and Excel reports; PowerPoint for the negotiation pack
- An administrator can upload a rate workbook
- 2,022 automatic software tests, all passing

NOT THERE YET

- Never compared against a price JLR has paid
- No history kept when someone changes a rate
- Finished costings are not saved, so you cannot go back and reproduce an old one
- No JLR sign-on; it has its own logins today
- Gear costing works on screen but the automated route cannot reach it
- Three faults found and fixed in August; we are still working on it

Why this is up front

Both options build on the left column and have to sort out the right one. I would rather you saw the right column now than found it later. Neither option gets to skip it.

WHY BOTHER AT ALL

What changes for the engineer

TODAY · typed in by hand

Part weight and starting material weight
Wall thickness and overall size
Counts of holes, pockets and bosses
Machine times for each operation
Surface area for painting or plating
Tolerances, finish, heat treatment

Two engineers costing the same part will not type
the same numbers. So we get two different answers.

WITH COSTVISION · read off the model

Hand over the 3D model and the drawing.

Measured off the model:

volume, size, surface area, wall thickness, holes,
pockets, bosses, gear teeth.

Read off the drawing:

tolerances, surface finish, heat treatment, coating,
masked features.

The same part gives the same numbers every time.

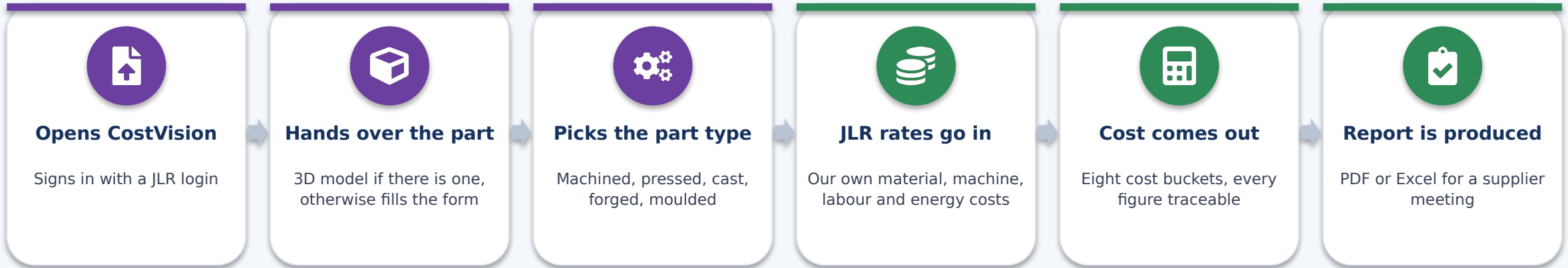
Measuring and reading are different things

The 3D model gets measured by software, much like a CMM measures a part. That is not AI. The AI only reads the words written on the drawing, and whatever it reads gets checked against the measured model before it can move a cost. Slide 9 covers those checks.

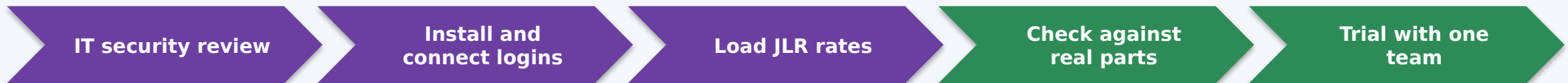
OPTION 2

Option 2: CostVision on JLR rates

What an engineer does each time they cost a part



What order the setup has to happen in



WE DO	Install on a JLR server	Connect JLR sign-on	Load the rates we are given	Run the accuracy comparison
JLR DOES	Security sign-off	Provide a server	Gather the rate data	Supply parts with real prices

The software is the easy bit

Gathering our own rate data sets the pace for everything else, and that work sits with us rather than with the tool.

OPTION 2 · DETAIL

What gets swapped for JLR numbers

STAYS THE SAME

- All 19 part types
- The costing maths
- Reading 3D models and drawings
- The safety checks
- PDF and Excel reports
- Design-for-cost findings

SWAPPED FOR JLR NUMBERS

- Material prices
- Machine rates per hour
- Labour rates by grade
- Electricity and gas prices
- Overhead and margin policy
- Rates for the countries we buy from

STILL TO BUILD · none of this exists yet

- JLR single sign-on
- A shared database
- AI routed through a JLR service
- History of every rate change
- Costings saved and auditable
- Backups to JLR standard

Changing the rates is a setup job

The tool was built so the rate book gets handed to it from outside rather than baked in, and there is already a screen for uploading one. Putting our numbers in does not change how anything is calculated.

Take the third column seriously

None of it exists today. Without the rate history and saved costings you cannot reproduce a costing from three months ago, and that rules the tool out for anything audited until it is built.

OPTION 2 · SEQUENCE

Option 2: order of work

1

IT security review

The data-flow document is already written. IT decide how they want the drawing-reading step handled.

2

Install and connect

Put it on a JLR server, connect JLR logins, move it onto a shared database.

3

Load JLR rates

Materials, machine rates, labour, energy, country factors. This one sets the pace and it is ours to do.

4

Check against real parts

Take 30 to 50 parts we have already bought and compare. This is how we find out how accurate it is.

5

Trial with one team

Real parts, run alongside CAPEE. Where the two disagree, work out why before going wider.

Step 3 sets the pace

Everything after it waits on our rate data.

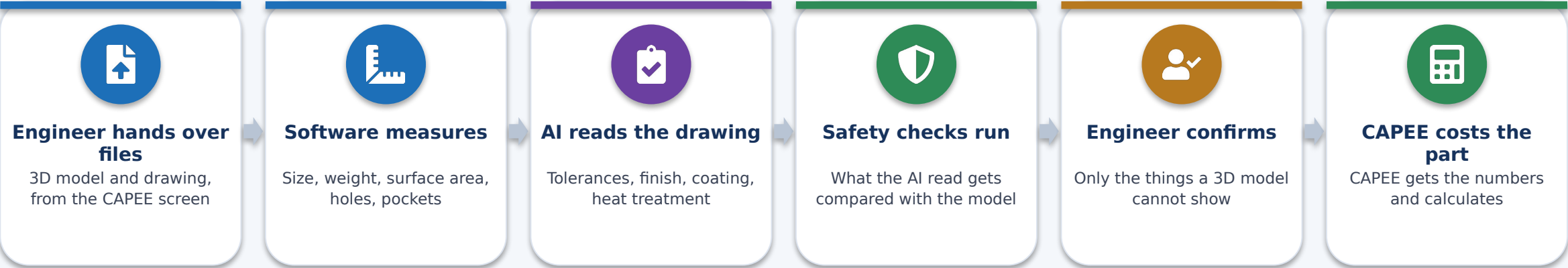
Step 4 answers the accuracy question

We get a measured number instead of a view.

OPTION 1

Option 1: automatic data into CAPEE

What happens each time an engineer costs a part



What order the build has to happen in



WE DO	Package the measuring software	Write out every setting	Move the safety checks across	Help with the CAPEE connection
JLR DOES	Tell us how CAPEE is built	Provide a server	Change CAPEE to take the numbers	Free up a team for the trial

One step we cannot size yet

Connecting into CAPEE depends on how CAPEE is built, and we do not know that yet. Slide 12 sets out the three ways of doing it.

OPTION 1 · REQUIRED

Safety checks

The AI does not set a price. It does hand over numbers that feed into one, so a bad reading turns into a bad cost. These four checks are what stop that happening, and they have to come across with everything else.



The measured model beats the AI

If the AI reads 2 kg off the drawing and the model measures 1.4 kg, we use the measured figure. The geometry wins.



Everything read gets cross-checked

Numbers taken off the drawing get compared against the measured model. Where they disagree, the engineer is told rather than the tool picking one.



Machining time has a ceiling

On a casting or forging, machining is capped at what finishing that part could realistically take, so a bad estimate cannot run the cost up.



Unknowns get asked, not guessed

Heat treatment, tolerance class and material grade cannot be read off a solid model, so the engineer is asked. The AI suggestion is labelled as a suggestion and never comes pre-ticked.

We hit this ourselves in August

Our review found a route through the tool with no checks on it. One missing input produced a cost of "not a number" while the software said it had worked fine. We have fixed it. It is also why I have put these checks in the scope.

OPTION 1 · HANDOVER

What we hand over

Sizes counted straight out of the software. Both languages are in common use and JLR IT will have people who know them.

Part	Language	Size	What it does
Measuring engine	Python	2,039 lines	Opens the 3D model and measures it. Makes no internet connection.
Reading the drawing	TypeScript	2,443 lines	Sends the drawing to the AI and sorts out what comes back.
Safety checks	TypeScript	536 lines	The four checks on the previous slide.
Questions per part type	TypeScript	7,037 lines	Works out what the engineer gets asked for each kind of part.
Design-for-cost findings	TypeScript	1,930 lines	Flags features that are expensive to make and says why.
Settings list	Plain list	one file	Every fixed number pulled together in one place. Next slide.

About the two languages

Python is the normal choice for engineering and measurement work. TypeScript is widely used for business software. Neither is unusual and neither ties JLR to one supplier.

One thing for IT to confirm

The measuring engine needs a normal Linux server, not the cut-down kind used for small services. It decides where this can go, so worth settling early.

OPTION 1 · SETTINGS

The fixed numbers in the code

These are engineering judgements written into the code. Six real ones, taken from the software as it stands:

What it controls	Value	Why
Smallest face counted as machined	400 mm²	Below this we treat it as an edge, not a face to machine
Smallest pocket counted	80 mm²	Stops small recesses getting costed as pockets
Cost of a rejected plated part	4.5x a good one	You strip it and plate it again, so a 3% reject rate is nearer 13% on cost
Zinc price used for plating	USD 3.67 / kg	Market reference, August 2026. Moves monthly and is meant to get updated
Shape allowance, pressing	1.15	A real pressing has more surface than a flat plate: edges, flanges, bends
Shape allowance, casting and forging	1.25 / 1.10	Castings carry ribs and bosses; forgings are simpler on the outside

How they get handed over

Pulled out of the code into one list you can read and change without a developer. That list is a deliverable in its own right and it is a step in the Option 1 sequence.

Where these came from

They are our engineering estimates. They are not measurements from a JLR plant. Treat them as a starting point and expect to replace them as real data comes in.

OPTION 1 • DECISION FOR IT

Three ways to connect it to CAPEE

We do not know yet how CAPEE is built. All three routes work. The answer decides which one we use and how much effort it takes.

Route	Use when	What it means day to day	Relative effort
A • Put it inside CAPEE	CAPEE is built in JavaScript	Our code goes straight in. Only the measuring engine sits separately.	Least effort
B • Run it alongside	CAPEE is Java, .NET or anything else	It runs as its own service on a JLR server and CAPEE asks it for numbers over a secure link.	Most effort
C • Pass a file	CAPEE is a desktop or spreadsheet tool	It writes a file that CAPEE imports. Simplest to build, but somebody has to move the file.	Least effort

If we had to pick blind

Route B. It works whatever CAPEE turns out to be, and we can update CostVision without touching CAPEE every time.

Three questions for IT

What is CAPEE written in. Can it call another service on the network. Can it run on a normal Linux server.

OPTION 1 · SEQUENCE

Option 1: order of work

1

Agree the connection

Find out how CAPEE is built, pick one of the three routes, agree exactly which numbers CAPEE expects.

2

Stand up the measuring software

Install it on a JLR server. Measure 20 real JLR parts and check the results against their drawings.

3

Hand over the settings list

Pull every fixed number into one list. JLR review it and sign it off before anything gets costed with it.

4

Move the safety checks across

The four checks and the questions the engineer confirms. In the scope from the start.

5

Connect into CAPEE

Wire it in. Route the drawing reading through a JLR-approved AI service. Cost one real part end to end.

Steps 1 and 2 can start together

Standing up the measuring software does not depend on knowing how CAPEE is built. Only step 5 does.

Start with one part type

Machined parts or pressings first. Most of our volume and the shapes we understand best. Widen it once it holds up.

COMPARISON

The two options side by side

	OPTION 1 · feed CAPEE	OPTION 2 · CostVision on JLR rates
Which system does the costing	CAPEE, unchanged	CostVision, beside CAPEE
Does the engineer learn a new tool	No, same CAPEE screens	Yes
How much software work	A lot	Not much
How much JLR data work	Some	A lot
Do we get the costing engine	No	Yes
Do we get design-for-cost findings	No	Yes
Does it tell us how accurate it is	No	Yes, step 4
Can we stop it part way	Hard, it sits inside CAPEE	Yes
Is it waiting on an unknown	Yes, how CAPEE is built	No

The last three rows are the important ones

Option 2 tells us how accurate the tool is, we can stop it, and it is not waiting on anyone. Option 1 does none of those three. That is why I would run Option 2 first.
No durations shown. Nothing in either option has been sized by JLR.

RECOMMENDATION

What I would do

Option 2 first, then Option 1

Option 2 is the smaller job, we can stop it whenever we want, and it answers the question Option 1 depends on: does the tool get the right number on JLR parts.



FIRST

Run Option 2

Install it, load our rates, compare against 30 to 50 parts we have already bought.

What we get: a real accuracy figure.



THEN

Decide

Look at the accuracy result and IT's answer on how CAPEE is built.

What we get: a decision based on evidence.



THEN

Run Option 1

Feed CAPEE automatically, starting with one part type.

What we get: more of something that already works.

If we can only fund one

Option 2. It is the smaller job, we can reverse it, it is not waiting on anyone, and it brings the costing engine and the design-for-cost findings with it. Option 1 mainly saves the engineer typing. Option 2 gives us something we do not have at all.

RISKS

Risks and open items

Item	Applies to	Detail and what sorts it	Rating
We do not know how accurate it is	Both	Never compared against a price JLR has paid. Step 4 of Option 2 sorts this.	High
Old costings cannot be reproduced	Option 2	No rate history and no saved costings. Has to be built before we use it for anything audited.	High
We do not know how CAPEE is built	Option 1	Holds up the choice of connection route. One answer from IT sorts it.	High
The built-in numbers are estimates	Both	Starting points rather than JLR plant measurements. They get replaced as real data comes in.	Medium
Server type	Option 1	The measuring engine needs a normal Linux server. Easy to confirm now, awkward if we miss it.	Medium
Drawing text goes to an AI service	Both	Short extracts only, and it can go through a JLR service. The CAD file itself stays on the server.	Medium
The tool is still changing	Both	Three faults found and fixed in August. Work is ongoing, so we pin a version for any trial.	Medium

One step deals with two of the three high items

Running Option 2 tells us how accurate the tool is and forces the rate history and saved costings to get built. Another reason to do it first.

NEXT STEPS

What we need to get going

-   **30 to 50 parts with the price we actually paid**
Part, description, annual volume, where it was made, and the price paid. Existing purchase records will do. This is how we find out how accurate it is, and both options need it.
-   **JLR rate data**
Material prices, machine rates per hour, labour by grade, electricity and gas. This sets the pace for Option 2, so it is worth starting whichever option we pick.
-   **How CAPEE is built**
What it is written in, whether it can call another service on the network, and what kind of server it runs on. Three questions to IT. Unblocks Option 1.
-   **A decision on the drawing-reading step**
Either switch off the AI reading and keep the measuring, or run it through a JLR-approved service. The data-flow document covers both.

The first one matters most

The other three can run alongside it. Without real prices to compare against, we cannot sign either option off as accurate.

SEQUENCE

What has to happen before what

No dates. None of this has been sized by JLR. What is fixed is the order, because some steps cannot start until others finish.



Can run alongside, holds nothing up

IT security review · Answer on how CAPEE is built · Standing up the measuring software · Writing out the settings list

We cannot check accuracy until JLR rate data is loaded.

I would not commit to Option 1 until we know how accurate it is.

We cannot scope the CAPEE connection until IT say how CAPEE is built.

Rate history and saved costings have to exist before we use it for anything audited.

APPENDIX

Where the numbers come from

Every count and value in this deck was read off the software in August 2026 and checked again when these slides were built.

Statement	How we checked it
19 part types can be costed	Traced each one from its screen through to the costing
The costing gives the same answer every run	Ran one part five times, identical to the last decimal
No hidden defaults in the costing	Every figure has to be supplied; nothing gets substituted
Measuring engine makes no internet connection	Checked every line for outbound calls; there are none
Measuring engine repeats exactly	Its one sampling step is fixed to a set value
CAD files stay on the server	Measuring happens locally; only drawing text goes to an AI service
Rates are given to the tool, not built in	The costing function takes the rate book as an input
Sizes on slide 10	Counted directly in the source files
Settings values on slide 11	Read off the software; each is a single defined value
Three faults found and fixed in August	A wrong-number result, a range that would not repeat, unreadable report text
2,022 automatic tests passing	Full test run at build time

Supporting documents

Security and data-flow review for IT · Technical readiness assessment · Independent review of the tool including a comparison against commercial alternatives.